Week 1

Environment Setup + Cowrie Honeypot

Ubuntu | Docker | Python | SSH Honeypot | Day 1-5

5 DAYS

GROUP — 4

PROJECT TEAM MEMBERS

#	Full Name	Role	Email ID	Phone
1	**Mitesh Chaudhari**	Security Engineer		
2		Backend Developer		
3		Frontend Developer / Dashboard		
4		DevOps / Documentation		

Project Information	
Project	Healthcare IoT Deception Honeypot Network
Organization	Infotact Solutions & Co.
Domain	Cybersecurity — IoT Deception Technology
Week	Week 1 of 4
Stack	Python + Flask + Docker + ELK Stack
Duration	5 Days (Day 1 to Day 5)
Version	v1.0.0

Week 1 Objectives

- Install Docker Engine and Docker Compose on Ubuntu
- Set up Python 3 virtual environment with all required packages
- Create complete project folder structure (honeypot, logging, dashboard, data)
- Configure and launch Cowrie SSH honeypot simulating a medical IoT patient monitor
- Verify JSON logs are being generated for all attack attempts

DAY 1 System Update + Docker Install

STEP 1 System Update

Goal: Keep Ubuntu updated

This command updates the list of available software packages from the internet and then upgrades all installed packages to their latest versions. The sudo keyword allows the command to run with administrative privileges, apt is the package manager used in Ubuntu, update refreshes the package list, and upgrade -y installs updates automatically without asking for user confirmation.

```
sudo apt update && sudo apt upgrade -y
```

STEP 2 Docker Dependencies

Goal: Required packages

This command installs essential packages required for secure communication and downloading files. The ca-certificates package ensures secure HTTPS connections, curl is used to download files from the internet, and gnupg is used to verify the authenticity of downloaded software using cryptographic keys. The next command creates a directory named keyrings inside /etc/apt with specific permissions. The -m 0755 sets read, write, and execute permissions for the owner and read-execute permissions for others, while -d specifies that a directory should be created.

```
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
```

STEP 3 Docker GPG Key + Repository

Goal: Official Docker signing key

This command downloads Docker's official GPG key securely using curl and then converts it into a format suitable for the system using gpg --dearmor. The output is stored in the specified location to ensure that Docker packages are verified and trusted.

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

This command adds the official Docker repository to the system's package sources. The echo command generates the repository configuration string, which includes system architecture and Ubuntu version, and tee writes this configuration into a file so that the system can download Docker packages from the official source.

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

STEP 4 Install Docker + Add User to Group

Goal: Docker engine install

The first command refreshes the package list again after adding the Docker repository, and the second command installs Docker Engine, Docker CLI, container runtime, and Docker Compose plugin, which are required to build and manage containers.

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

This command adds the current user to the Docker group so that Docker commands can be executed without using sudo. The newgrp docker command applies the group changes immediately in the current session.

```
sudo usermod -aG docker $USER
newgrp docker
```

These commands are used to verify that Docker and Docker Compose have been successfully installed by displaying their respective versions.

```
docker --version
docker compose version
```

Expected Output:

```
Docker version 24.x.x
Docker Compose version v2.x.x
```

NOTE: After newgrp docker, stay in the same terminal

STEP 5 — Install Python 3

Goal: Python runtime

This command installs Python 3 along with its package manager (pip), virtual environment tool (venv), and Git, which is used for version control and managing project code.

```
sudo apt install -y python3 python3-pip python3-venv git
```

Expected Output:

```
Python 3.10.x
```

DAY 2 — Project Structure + Python Environment

STEP 6 — Create Project Folder + All Subfolders

Goal: Complete directory structure

This command creates a new directory named healthcare-iot-honeypot in the user's home directory and then navigates into that directory using the cd command.

```
mkdir -p ~/healthcare-iot-honeypot
cd ~/healthcare-iot-honeypot
```

This command creates multiple directories at once using brace expansion, which helps in setting up a structured folder hierarchy required for the honeypot filesystem.

```
mkdir -p honeypot/cowrie/filesystem/{etc,home/nurse,var/log}
mkdir -p logging/{filebeat,logstash/pipeline,parsers}
mkdir -p dashboard/{templates,static/css,static/js}
mkdir -p data/{logs/cowrie,processed,malware/quarantine,iocs}
mkdir -p analysis/{reports,threat_intel} scripts docs
```

STEP 7 — Python Virtual Environment + Packages

Goal: Isolated Python environment

This command creates a virtual environment named venv using Python, which isolates project dependencies from the global system environment. The source command then activates the virtual environment so that any Python packages installed afterward will be confined to this

environment only. The pip upgrade command upgrades the Python package manager pip to the latest version to ensure compatibility and improved performance.

```
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
```

This creates the requirements file that lists all the Python libraries required for the project. Each package includes a specific version number to ensure compatibility and consistent behavior across different environments.

```
cat > requirements.txt << 'EOF'
flask==2.3.3
flask-socketio==5.3.4
eventlet==0.33.3
elasticsearch==8.11.0
pandas==2.1.3
requests==2.31.0
geoip2==4.7.0
gunicorn==21.2.0
EOF
```

This command installs all the Python libraries listed in the requirements.txt file, ensuring that all dependencies required for the project are available.

```
pip install -r requirements.txt
```

NOTE: This will take a few minutes to download all packages

DAY 3 Cowrie Configuration Files

STEP 8 Create Cowrie Dockerfile

Goal: Container build instructions

This command creates a Dockerfile and allows the user to input multiple lines of configuration until EOF is encountered, which defines how the Docker container should be built. The COPY instructions copy files from the local system into the Docker container so that the honeypot has access to configuration files and fake credentials. The EXPOSE instruction specifies that the container will listen on ports 2222 (SSH) and 2223 (Telnet), allowing external connections to the honeypot.

```
cat > honeypot/cowrie/Dockerfile << 'EOF'
FROM cowrie/cowrie:latest
COPY userdb.txt /cowrie/cowrie-git/etc/userdb.txt
COPY filesystem/home /cowrie/cowrie-git/share/cowrie/honeyfs/home
COPY filesystem/etc /cowrie/cowrie-git/share/cowrie/honeyfs/etc
COPY filesystem/var /cowrie/cowrie-git/share/cowrie/honeyfs/var
EXPOSE 2222 2223
EOF
```

STEP 9 — Create cowrie.cfg + userdb.txt

Goal: Honeypot config and credentials

This command creates the configuration file for the honeypot, where parameters like hostname, ports, and logging settings are defined.

```
cat > honeypot/cowrie/cowrie.cfg << 'EOF'
[honeypot]
hostname = patient-monitor-03
ssh_port = 2222
telnet_port = 2223

[output_jsonlog]
enabled = true
logfile = /var/log/cowrie/cowrie.json
EOF
```

This command creates a file containing fake usernames and passwords that attackers can use to attempt login into the honeypot.

```
cat > honeypot/cowrie/userdb.txt << 'EOF'
root:0:root
root:0:123456
admin:0:admin
admin:0:password
nurse:0:nurse
doctor:0:doctor
guest:0:guest
EOF
```

NOTE: Format: username:uid:password

DAY 4 — Fake Medical Files + Docker Compose

STEP 10 — Create Fake Patient Records + Device Files

Goal: Deception content

This command creates a file containing fake patient records, which is used to make the honeypot appear realistic to attackers.

```
cat > honeypot/cowrie/filesystem/home/nurse/patient_records.txt << 'EOF'
PATIENT RECORDS - STRICTLY CONFIDENTIAL
========================================
Room 302: John Smith      | HR: 72 | BP: 120/80 | O2: 98%
Room 304: Maria Garcia    | HR: 68 | BP: 118/76 | O2: 99%
Room 315: James Wilson    | HR: 92 | BP: 142/92 | ALERT: arrhythmia
EOF
```

This command writes the text patient-monitor-03 into a file, setting the hostname of the simulated IoT device. The second command sets the login banner to display a fake medical device name, making the honeypot appear as a real CareFusion Patient Monitor to any attacker who connects.

```
echo 'patient-monitor-03' > honeypot/cowrie/filesystem/etc/hostname
echo 'CareFusion Patient Monitor v2.3.1' > honeypot/cowrie/filesystem/etc/issue.net
```

STEP 11 Create docker-compose.yml

Goal: All services definition

This command creates the Docker Compose configuration file that defines all services, networks, and volumes for the honeypot system. It sets up a dedicated network with a fixed subnet, maps the honeypot ports externally, and binds the log directory so attack logs are accessible from the host.

```
cat > docker-compose.yml << 'EOF'
networks:
  honeypot_net:
    driver: bridge
    ipam:
      config:
        - subnet: 192.168.100.0/24
  monitoring_net:
    driver: bridge

services:
  cowrie:
    build: ./honeypot/cowrie
    container_name: cowrie_honeypot
    networks:
      honeypot_net:
        ipv4_address: 192.168.100.10
    ports:
      - '2222:2222'
      - '2223:2223'
    volumes:
      - ./data/logs/cowrie:/var/log/cowrie
    restart: unless-stopped

volumes:
  es_data:
EOF
```

NOTE: Do NOT add version: field — it is obsolete in newer Docker

DAY 5 Cowrie Start + Test

STEP 12 Start Cowrie + SSH Test + Verify Logs

Goal: Launch and validate honeypot

The docker compose up -d cowrie command starts the Cowrie honeypot container in detached mode, meaning it runs in the background without blocking the terminal. The docker compose ps command then displays the status of running Docker containers to verify that the honeypot is active.

```
docker compose up -d cowrie

docker compose ps
```

This command removes an old SSH key entry from the known hosts file to avoid connection errors when reconnecting to the honeypot on the same port.

```
ssh-keygen -f '/home/$USER/.ssh/known_hosts' -R '[localhost]:2222'
```

This command connects to the honeypot via SSH using port 2222 with the username root, simulating an attacker trying to access the system. After login, commands like whoami and ls can be run inside the fake environment to generate log entries.

```
ssh -p 2222 -o StrictHostKeyChecking=no root@localhost

# Password: root — then type: whoami, ls, exit
```

This command retrieves the last 5 lines from the honeypot log file and formats the JSON data into a readable structure using Python, confirming that attack attempts are being logged correctly.

```
tail -5 data/logs/cowrie/cowrie.json | python3 -m json.tool
```

Expected Output:

```
cowrie_honeypot   Up
root@svr04:~# whoami
root
```

STEP 13 start-honeypot.sh — Auto Start Script

Goal: Ek command se sab start

This script automates the startup of the Healthcare IoT Honeypot system. It navigates to the project directory, removes any existing dashboard container to avoid conflicts, and starts all services using Docker Compose in detached mode. It also ensures that logging components are properly initialized and finally launches the dashboard for monitoring attacks.

```
# nano editor
```

```
nano ~/start-honeypot.sh
```

```
# Script content
```

```
#!/bin/bash

cd ~/healthcare-iot-honeypot

echo 'Starting Healthcare IoT Honeypot...'

docker rm -f attack_dashboard 2>/dev/null

docker compose up -d

echo 'Waiting 30 seconds for Logstash...'
```

```
sleep 30

LOGSTASH_IP=$(docker inspect logstash | grep '"IPAddress"' | grep '192' | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | head -1)

sudo tee /home/mitesh/healthcare-iot-honeypot/logging/filebeat/filebeat.yml > /dev/null << FBEOF

filebeat.inputs:

  - type: log

    enabled: true

    paths:

      - /var/log/honeypot/cowrie/*.json

    json.keys_under_root: true

    json.overwrite_keys: true

    tags: ["cowrie"]


output.logstash:

  hosts: ["${LOGSTASH_IP}:5044"]

FBEOF

sudo chown root:root /home/mitesh/healthcare-iot-honeypot/logging/filebeat/filebeat.yml

sudo chmod go-w /home/mitesh/healthcare-iot-honeypot/logging/filebeat/filebeat.yml

docker compose restart filebeat

sleep 15

ES_IP=$(docker inspect elasticsearch | grep '"IPAddress"' | grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' | tail -1)

docker rm -f attack_dashboard 2>/dev/null

docker run -d --name attack_dashboard --network healthcare-iot-honeypot_monitoring_net -p 5000:5000 -e ELASTICSEARCH_HOST=http://${ES_IP}:9200 -e SECRET_KEY=change-me -v /home/mitesh/healthcare-iot-honeypot/data/processed:/app/data/processed healthcare-iot-honeypot-dashboard

echo 'All Done! Dashboard: http://localhost:5000'
```

 # Save + make executable

```
# Ctrl+O -> Enter -> Ctrl+X

chmod +x ~/start-honeypot.sh

echo "alias honeypot='bash ~/start-honeypot.sh'" >> ~/.bashrc

source ~/.bashrc
```

STEP 14	auto-attacks.sh — Test Attack Script	Goal: Automatic test data generator

This script simulates multiple cyberattacks on the honeypot by sending fake data to Elasticsearch. It generates different types of attacks such as brute force, credential theft, command injection, and malware delivery from various locations. The script helps in testing the system by populating the dashboard with sample attack data.

 # nano

```
nano ~/auto-attacks.sh
```

 # Script — bruteforce China

```
for i in 1 2 3 4 5; do

  curl -s -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" \

    -H 'Content-Type: application/json' \

    -d "{\"@timestamp\":\"$(date -u +%Y-%m-
%dT%H:%M:%S.000Z)\",\"eventid\":\"cowrie.login.failed\",\"attack_type\":\"bruteforce\",\"src_ip\"
:\"1.180.0.$i\",\"username\":\"admin\",\"password\":\"admin\",\"geoip\":{\"country_name\":\"China
\",\"location\":{\"lat\":39.9042,\"lon\":116.4074}}}"

  sleep 1

done
```

 # Script — other attack types

```
# credential_theft - Russia

curl -s -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type:
application/json' -d "{\"@timestamp\":\"$(date -u +%Y-%m-
%dT%H:%M:%S.000Z)\",\"eventid\":\"cowrie.login.success\",\"attack_type\":\"credential_theft\",\"s
rc_ip\":\"5.8.0.1\",\"username\":\"root\",\"password\":\"root\",\"geoip\":{\"country_name\":\"Rus
sia\",\"location\":{\"lat\":55.7558,\"lon\":37.6173}}}"

# command_injection - USA

curl -s -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type:
application/json' -d "{\"@timestamp\":\"$(date -u +%Y-%m-
%dT%H:%M:%S.000Z)\",\"eventid\":\"cowrie.command.input\",\"attack_type\":\"command_injection\",\"
src_ip\":\"8.8.0.1\",\"username\":\"root\",\"geoip\":{\"country_name\":\"United
States\",\"location\":{\"lat\":40.7128,\"lon\":-74.0060}}}"

# malware_delivery - India

curl -s -X POST "http://localhost:9200/honeypot-$(date +%Y.%m.%d)/_doc" -H 'Content-Type:
application/json' -d "{\"@timestamp\":\"$(date -u +%Y-%m-
%dT%H:%M:%S.000Z)\",\"eventid\":\"cowrie.session.file_download\",\"attack_type\":\"malware_delive
ry\",\"src_ip\":\"103.21.0.1\",\"username\":\"admin\",\"geoip\":{\"country_name\":\"India\",\"loc
ation\":{\"lat\":19.0760,\"lon\":72.8777}}}"

echo 'Done! Browser me F5 press karo!'
```

 # Save + alias

```
chmod +x ~/auto-attacks.sh

echo "alias attacks='bash ~/auto-attacks.sh'" >> ~/.bashrc

source ~/.bashrc
```

Completion Checklist
✓ docker --version and docker compose version work
✓ cowrie_honeypot container Up on port 2222
✓ SSH login works: ssh -p 2222 root@localhost (password: root)
✓ data/logs/cowrie/cowrie.json contains JSON events
✓ start-honeypot.sh script created and working
✓ auto-attacks.sh script created and working
✓ Aliases set: honeypot and attacks commands work
✓ Prometheus config file created: monitoring/prometheus/prometheus.yml

Week 1 — Required GitHub Commits

Commit Message	Files Changed
`git commit -m "feat: init project structure and docker setup"`	`docker-compose.yml, README.md`
`git commit -m "feat: add Cowrie honeypot Dockerfile"`	`honeypot/cowrie/Dockerfile`
`git commit -m "feat: add cowrie.cfg with medical device config"`	`honeypot/cowrie/cowrie.cfg`
`git commit -m "feat: add fake credentials userdb.txt"`	`honeypot/cowrie/userdb.txt`
`git commit -m "feat: add fake patient records deception files"`	`honeypot/cowrie/filesystem/home/nurse/patient_records.txt`
`git commit -m "feat: add requirements.txt and Python venv"`	`requirements.txt, venv/`
`git commit -m "fix: add prometheus.yml config file"`	`monitoring/prometheus/prometheus.yml`
`git commit -m "feat: add start-honeypot.sh auto start script"`	`~/start-honeypot.sh`

Commit Message	Files Changed
`git commit -m "feat: add auto-attacks.sh test data script"`	`~/auto-attacks.sh`
`git commit -m "test: verify SSH login works on port 2222"`	`data/logs/cowrie/cowrie.json`